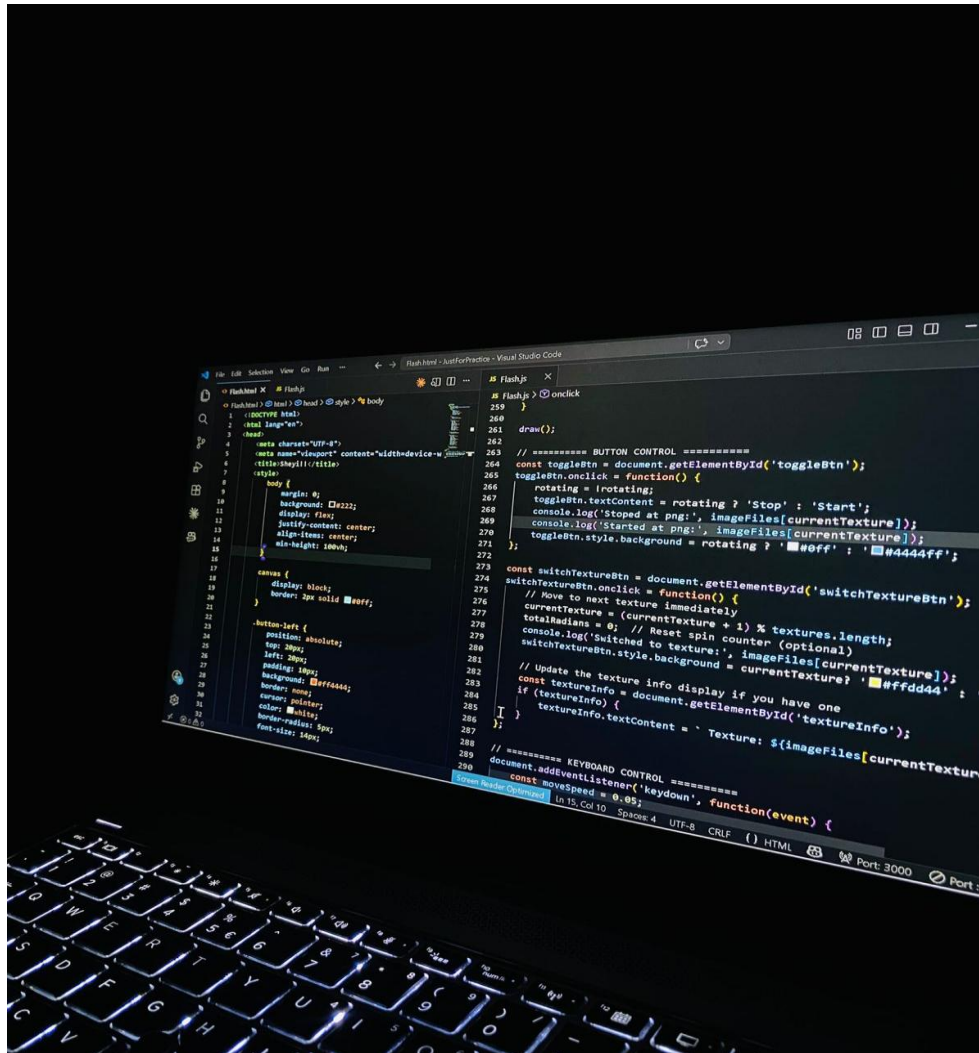




SPACE SHOOTER GAME

WEBGL
IMPEMENTATION
DATE: 04 MAY 2026
STUDENT NUMBER:
241164907
NAME:
SIMAMKELE
SHEYI
COURSE:
COMPUTER
GRAPHICS
PROJECT

1. INTRODUCTION

This report documents a 3D space shooter game built using WebGL 1.0 for a computer graphics assignment. The objective was to create an interactive game demonstrating core OpenGL concepts: matrix transformations, perspective projection, shader programming, and vertex buffer management. The game was developed from scratch using pure JavaScript and GLSL shaders with no external libraries. It consists of two files: index.html (HTML/CSS) and space-shooter.js (WebGL and game logic).

The player controls a starfighter at the bottom of the screen, shooting waves of descending enemy ships. The game features three enemy types, a triple-shot weapon, particle explosions, procedurally generated sound effects, and a progressive wave system. It runs at approximately 60 FPS using requestAnimationFrame.

2. GAME INFORMATION

Game Name: Space Shooter

Game Rules: The player starts with 3 lives and must destroy enemies while avoiding being hit. Points: basic enemies (red) = 10, fast enemies (orange) = 15, tough enemies (pink) = 25. A life is lost when an enemy reaches the bottom or collides with the player. After losing a life, the player gets 1.5 seconds of invincibility with the ship blinking. The game ends when all lives are lost.

Waves increase difficulty every 8 seconds. Enemies spawn faster and move quicker with each wave. Fast enemies appear from wave 3, tough enemies from wave 5. The player fires 3 bullets in a spread pattern using Spacebar. Controls: Arrow Keys/WASD to move, Spacebar to shoot.

3. HOW TO COMPILE AND RUN

No compilation is required. The game runs directly in any modern web browser. To run: place both files in the same folder, open index.html in Chrome, Firefox, or Edge, and click "LAUNCH MISSION." Alternatively, use a local server: `python -m http.server 8000` then open `http://localhost:8000`. Requirements: a modern browser with WebGL 1.0 support and JavaScript enabled. No build tools, compilers, or dependencies needed.

4. DESIGN OVERVIEW

4.1 Architecture and Rendering

The project separates presentation (index.html with CSS, HUD, screens) from logic (space-shooter.js with WebGL engine, game systems). The renderer uses a painter's algorithm with depth testing disabled. Objects draw in fixed order: stars ($z = -2.5$), particles, bullets, enemies, then player ship ($z = -2.0$). Each draw call builds a transformation matrix (translate, optionally rotate, optionally scale) and sends it to the GPU with vertex color data.

4.2 Game Loop and State Machine

requestAnimationFrame drives the loop at 60 FPS. Each frame: clears the buffer, processes input, moves objects, checks collisions, spawns enemies, updates HUD, draws all objects in layer order. Three states: START (title screen with floating ship), PLAYING (all systems active), GAMEOVER (final score with restart option).

4.3 Entity and Collision Design

Game objects use simple JavaScript objects rather than classes. Player stores position, speed, invincibility, engine glow. Enemies store position, type, speed, health, wobble. Bullets store position only. Particles store position, velocity, life, color. Stars store position, size, brightness, twinkle parameters.

Collision detection uses circle-based Euclidean distance. Bullet vs enemy: 0.1 unit radius. Player vs enemy: 0.13 unit radius. All loops iterate backwards for safe element removal during iteration.

4.4 Difficulty and Audio

Waves last 500 frames. Spawn delay decreases from 40 to minimum 15 frames. Enemy speed increases 0.0008 per wave. Enemy probabilities: 50% basic, 33% fast, 17% tough.

Audio uses Web Audio API square wave oscillators generated at runtime. Frequencies: shooting 880Hz, enemy destroyed 400Hz, tough enemy first hit 600Hz, player hit 100Hz, player death 50Hz, wave change 660Hz and 880Hz. Audio initializes on first button click for browser compliance.

4.5 Visual Design

Retro arcade aesthetic. Player ship: 18 vertices forming 6 colored components (cyan fuselage, light blue cockpit, cyan wings, orange engines). Enemies: 9-vertex diamond with antenna, colors distinguish types (red/orange/pink). Background: 200 stars with individual twinkling. CSS adds CRT scanlines, vignette, neon glow, flicker effects. All graphics use vertex colors without textures.

5. WEBGL CONCEPTS IMPLEMENTED

5.1 Initialization and Context

The WebGL context is obtained via `canvas.getContext('webgl')`. If unsupported, an alert displays and execution stops. Clear color is black. Depth testing is disabled because the painter's algorithm handles layering by draw order, and all game objects share $z = -2.0$.

5.2 Matrix Transformations

Five matrix functions were implemented from scratch as `Float32Array(16)` in column-major order.

Identity: Starting point for all transformation chains. Ones on diagonal, zeros elsewhere.

Translation: Positions objects via tx, ty, tz in the fourth row. Used for player movement, enemy descent, bullet travel, particle spread. Game objects at $z = -2.0$, stars at $z = -2.5$.

Scale: Resizes objects via sx, sy, sz on diagonal. Tough enemies 1.4x larger. Particles shrink as they fade via `scale(alpha, alpha, 1.0)`. Stars vary 0.2-0.9 size. Engine glow pulses via sine wave: `scale(0.5glow, 0.8glow, 1.0)`.

Rotation (Z-axis): Available but currently unused. Uses cos/sin pattern for 2D rotation. Defined for future use.

Multiplication: Combines transformations in TRS order (Translate, Rotate, Scale) using three nested loops (64 operations per multiply). Ensures objects scale/rotate around local origin before positioning.

5.3 Perspective Projection

Creates depth illusion using 45-degree FOV, aspect ratio $800/600 = 1.33$, near plane 0.1, far plane 100.0. Calculates focal length $f = 1/\tan(\text{fov}/2)$ and depth factor $nf = 1/(\text{near} - \text{far})$. Stars at $z = -2.5$ appear smaller than objects at $z = -2.0$ due to automatic perspective division by the GPU. Projection matrix set once at initialization, uploaded as `uProjection` uniform.

5.4 Shader Programming

Vertex Shader: Attributes `aPosition(vec3)` and `aColor(vec3)`. Uniforms `uProjection(mat4)` and `uModel(mat4)`. Transforms vertices: $gl_Position = uProjection * uModel * vec4(aPosition, 1.0)$. Passes color to fragment shader via `vColor` varying. GPU interpolates colors across triangles.

Fragment Shader: Uses mediump precision for device compatibility. Outputs $gl_FragColor = vec4(vColor, 1.0)$. No textures used - all coloring from vertex color arrays.

Compilation: createShader function compiles GLSL, checks errors with glGetShaderInfoLog(), returns null on failure. Both shaders linked into a program activated with useProgram().

5.5 Vertex Buffer Objects

Five geometry buffers store shapes in GPU memory:

Five geometry buffers store the shape data in GPU memory. The ship buffer is the largest, containing 18 vertices organized into six triangle groups that form the player starfighter (main fuselage, cockpit, two wings, and two engines), using 216 bytes of memory. The enemy buffer holds 9 vertices forming a diamond-shaped body with an antenna made from three triangles, using 108 bytes. This single buffer is shared by all three enemy types, with colors and scale applied at render time to distinguish them. The bullet buffer contains 6 vertices forming a small elongated rectangle from two triangles, using 72 bytes. The star buffer also uses 6 vertices forming a small square from two triangles, also 72 bytes, and is reused 200 times for the background starfield with each star's apparent size set through scaling.

The particle buffer similarly uses 6 vertices and 72 bytes for a small square, shared by both explosion particles and engine glow effects. In total, all five geometry buffers contain 45 vertices forming 15 triangles, using only 540 bytes of GPU memory. This efficient use of shared geometry keeps the game running smoothly even with many objects on screen. All geometry buffers use STATIC_DRAW (uploaded once, drawn many times). Color buffers use DYNAMIC_DRAW for per-frame updates (enemy colors by type, star brightness twinkling, particle alpha fading). Ship color buffer uses STATIC_DRAW since its multi-colored design never changes.

5.6 Rendering Pipeline

The drawShape function: binds position buffer, sets attribute pointer (3 floats), uploads color data via DYNAMIC_DRAW, builds model matrix (identity → translate → optional rotate → optional scale), uploads to uModel uniform, draws with drawArrays(TRIANGLES).

Render order: stars (brightness = brightness + sin(time * speed * 100 + phase) * 0.25, min 0.15), particles (alpha = life/maxLife), bullets (yellow), enemies (color-coded, tough scaled 1.4x), player ship (blinks during invincibility: floor(invincible/5) % 2 == 0), engine glow (orange, pulsing scale).

6. GAME SYSTEMS

6.1 Player System

Player tracks x, y, speed (0.07), invincibility, engine glow. Input checks keys object supporting Arrow Keys and WASD. Diagonal movement normalized by 0.707 to prevent speed advantage. Position clamped between -0.9/0.9 and -0.85/0.85. After damage, 90 frames (1.5s) invincibility with blinking feedback.

6.2 Enemy System

Spawns at enemyDelay intervals. Weighted random: 50% basic, 33% fast, 17% tough. 30% chance of double spawn. Fast enemies multiply speed by 1.8, zigzag via wobble ± 0.08 , $x \pm \sin(\text{wobble}) * 0.004$. Tough enemies multiply speed by 0.6, scale 1.4x, health 2. Enemies below $y = -1.1$ remove themselves; if player not invincible, life lost.

6.3 Bullet and Particle Systems

Spacebar fires 3 bullets in spread: center (x, y+0.1), sides ($x \pm 0.05$, y+0.06). Shoot timer 6 frames (0.1s cooldown). Bullets move at 0.13 speed; removed when $y > 1.2$.

Explosions spawn particles in circular burst: 20 for basic/fast, 32 for tough. Max 200 particles. Each has random angle velocity, 1.5s life decreasing 0.03/frame, orange color with variation. Alpha = life/maxLife for fade effect; scale = alpha for shrink effect.

6.4 Collision Detection

Euclidean distance: $\sqrt{dx^2 + dy^2}$. Bullet vs enemy: 0.1 radius. On hit, bullet removed, enemy health--. At zero health: explosion, score increased, enemy removed. Tough enemies play 600Hz on first hit, 400Hz on kill. Player vs enemy: 0.13 radius, only when not invincible. On hit: explosion, enemy removed, life lost, 90 frame invincibility, 100Hz sound. All loops reverse-iterated.

6.5 Wave System

Wave counter increments each frame. At 500 frames: wave++, counter reset, enemyDelay = $\max(15, 40 - \text{wave} * 2)$. Minimum delay 15 frames at wave 13. Two ascending tones play (660Hz, 880Hz after 150ms). Enemy base speed: $0.007 + \text{wave} * 0.0008$.

7. PERFORMANCE AND LIMITATIONS

Performance maintained through: buffers created once (540 bytes total GPU memory), DYNAMIC_DRAW color updates (no new buffer allocation), 200 particle cap, immediate removal of off-screen objects, reverse iteration for single-pass cleanup, 800x600 resolution, simple geometry (max 18 vertices per object).

Limitations: no pause feature, no high score saving, no boss enemies, audio requires initial button click, keyboard-only controls, infinite difficulty scaling with no ceiling.

8. SOURCE CODE

Two files provided:

index.html: HTML structure, CSS styling (CRT scanlines, neon glow, vignette, flicker animations), HUD overlay (score/lives/wave), start screen with launch button, game over screen with restart button, controls display.

space-shooter.js: WebGL initialization, five matrix functions (identity, translate, scale, rotz, multiply), perspective projection, vertex/fragment shaders with compilation, five geometry buffers (STATIC_DRAW), five color buffers (DYNAMIC_DRAW), game state machine (start/playing/gameOver), player movement with diagonal normalization, enemy spawning with weighted random and zigzag AI, triple-shot bullet system with cooldown, particle explosion system with alpha fade, collision detection with distance formula, wave progression with increasing difficulty, Web Audio API sound generation (square wave oscillators), render loop via requestAnimationFrame at 60 FPS.

No compiled binaries needed. No dependencies or build tools required.

9. CONCLUSION

This project demonstrates core WebGL concepts through a complete interactive game. Implemented features include: custom matrix transformations (translate, scale, rotate, multiply), perspective projection (45° FOV), GLSL vertex and fragment shaders, vertex buffer objects (STATIC_DRAW geometry, DYNAMIC_DRAW colors), vertex coloring pipeline, requestAnimationFrame game loop at 60 FPS, collision detection via distance formula, particle system with life-based alpha fading, procedurally generated audio, and wave-based difficulty progression. All features integrate into a playable space shooter applying graphics concepts from the course.

10. REFERENCES

- WebGL Fundamentals. <https://webglfundamentals.org/>
- Course lecture notes on matrix transformations, perspective projection, and the graphics pipeline.